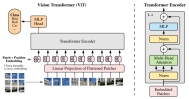
Advanced Artificial Intelligence and Machine Learning

Transformers Outside NLP



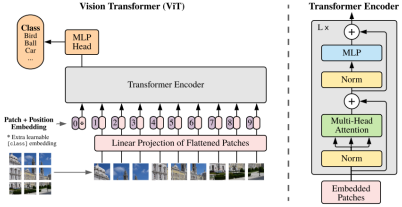
Learning goals

- Vision Transformers: images as patch sequences, inductive bias, transfer learning; Swin's shifted windows
- DETR and MaskFormer: detection and segmentation with a Transformer decoder and set-based loss
- Knowledge distillation for vision models
- Vision language models, contrastive learning, CLIP, BLIP and OWL-ViT
- Video Transformers: ViViT and TimeSformer

Vision Transformers for Image Classification

VISION TRANSFORMER (VIT)

- As the Transformer architecture scaled well in NLP, the same architecture was applied to images by creating small patches of the image and treating them as tokens: the **Vision Transformer (ViT)**.
- Images are reorganized as 2D grids of patches; the model is trained on those patches.



▸ Source: Dosovitskiy et al., 2020

CNNS VS. VISION TRANSFORMERS: INDUCTIVE BIAS

- **Inductive bias**: the set of assumptions a learning algorithm uses to make predictions — a shortcut that helps a model make educated guesses based on what it has seen so far.
- Two inductive biases of CNNs:
  - **Translational equivariance**: an object can appear anywhere in the image, and CNNs can detect its features.
  - **Locality**: pixels interact mainly with their surrounding pixels to form features.
- ViT does not have these assumptions. That is why, up to a certain dataset size, CNNs are actually better than ViT.
- But: being (mostly) different types of linear functions, the Transformer makes ViT highly scalable — which lets ViT overcome the missing inductive biases with massive amounts of data.

TRANSFER LEARNING & FINE-TUNING

- Training a ViT on millions of images is not feasible for everyone. Instead, use openly available pre-trained weights (e.g. the Hugging Face Hub).
- **Transfer learning**: leverage the features a ViT learned on a very large dataset and apply them to our (possibly small) dataset.
  - By freezing most of the weights, we train only certain layers: good performance with less training time and low GPU consumption.
- **Fine-tuning**: if our domain differs from the pre-training data, update the weights of the entire pre-trained model, with a lower learning rate.
- In most scenarios, transfer learning is ample for Vision Transformers.
- Both work for **multi-class** (one label per image) and **multi-label** (several labels per image) classification.

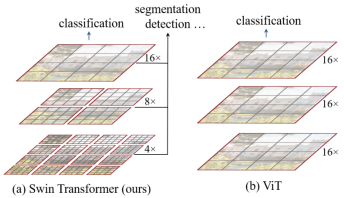
Swin Transformer

SWIN TRANSFORMER

- Introduced in 2021 Liu et al., 2021: optimizes for latency and performance using a **shifted window** (as opposed to sliding window) approach, which reduces the number of operations required.
- Swin is a **hierarchical backbone** for computer vision, e.g. for image classification.
  - A *backbone* is the part of a network that does feature extraction; additional layers can be added on top for a variety of vision tasks.
  - *Hierarchical* backbones have tiered structures, sometimes with varying resolutions — in contrast to the non-hierarchical *plain* backbone of e.g. ViTDet.

SHIFTED WINDOWS

- In the original ViT, attention is done between each patch and all other patches — computationally intensive.
- Swin reduces ViT's quadratic complexity to **linear** complexity (w.r.t. image size): similar to a CNN, patches only attend to other patches in the **same window**, and are then gradually merged with neighboring patches. This is what makes Swin hierarchical.



▸ Source: Liu et al., 2021

SWIN: ADVANTAGES AND SUCCESSORS

- **Computational efficiency**: Swin is more performant than completely patch-based approaches like ViT.
- **Large datasets**: as training size goes up, Swin outperforms CNNs; the large number of parameters enables increased capacity for learning more complex representations.
- **Swin Transformer V2** Liu et al., 2022: one of the first 3B-parameter vision models, trainable with high-resolution images. It stabilizes training, transfers models pre-trained on low-resolution images to high-resolution tasks, and uses SimMIM, a self-supervised approach that reduces the number of labeled images required.
- **Image restoration**: SwinIR turns low-resolution into high-resolution images; Swin2SR improves on it using Swin V2.

SWIN IMPLEMENTATION: THE MODEL

Key parts of the original PyTorch implementation:

1. **Parameters**, among others: `window_size` (size of the windows for local self-attention), `ape` (add absolute position embeddings?).
2. **Patch embedding**: similar to ViT, the image is split into non-overlapping patches, linearly embedded using a `Conv2D`.
3. **Positional embeddings** (optional, `ape`): added to the patch embeddings to help the model use positional information about each patch.
4. **Stochastic depth decay**: regularization by skipping layers during training — the deeper the layer, the higher the chance it is skipped.
5. **Layers**: multiple `BasicLayers` of `SwinTransformerBlocks`, each downsampling the feature map via `PatchMerging`. Feature dimensionality and resolution change across layers.
6. **Classification head**: as in ViT, an MLP head.

SWIN IMPLEMENTATION: THE BLOCK

Components: LayerNorm → **window attention** (self-attention within local windows) → drop path (stochastic depth) → LayerNorm → MLP; an **attention mask** controls which elements in the windowed input may attend to each other.

Forward pass, four steps:

1. **Cyclic shift**: the feature map is partitioned into windows, and the partitions are cyclically shifted (e.g. `A, B, C, D` shifted right by one becomes `D, A, B, C`). This lets the model capture relationships between adjacent windows.
2. **Windowed attention**: window-based multi-head self-attention (W-MSA).
3. **Merge patches** via `PatchMerging`.
4. **Reverse cyclic shift**: window partitioning and shift are undone, so the feature map retains its original form.

WINDOW ATTENTION AND PATCH MERGING

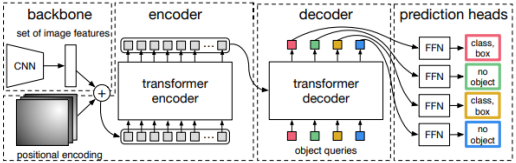
- **Window attention**: window-based multi-head self-attention (W-MSA) with relative position bias; used for both shifted and non-shifted windows.
- **Patch merging**: downsampling. Reduces the spatial dimensions of the feature map, similar to pooling in CNNs — building hierarchical feature representations by progressively increasing the receptive field and reducing the spatial resolution.

DEtection TRansformer (DETR)

OBJECT DETECTION WITH DETR

- **Object detection**: detect objects in an image. Input: e.g. an image of a road. Output: e.g. `[('car',X1,Y1,W1,H1), ('pedestrian',X2,Y2,W2,H2)]` — class, plus the x, y coordinates, width and height of each bounding box.
- Traditional detectors like YOLO use hand-crafted features such as **anchor box priors** (initial guesses of object locations and shapes), and post-processing steps to remove overlapping boxes, which need carefully selected filtering heuristics.
- **DETR** ▸ Carion et al., 2020 simplifies the detector: an encoder-decoder Transformer after the feature-extraction backbone **directly predicts bounding boxes in parallel**, requiring minimal post-processing.

DETR: ARCHITECTURE

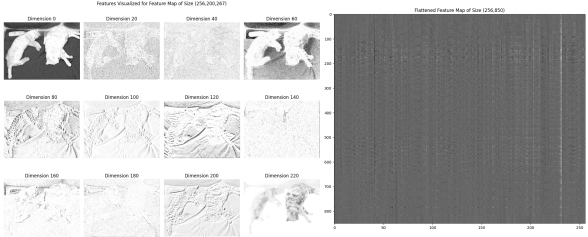


Source: Carion et al., 2020

- A CNN backbone; its output is fed into a Transformer encoder, resulting in N embeddings.
- The decoder uses learned positional embeddings (**object queries**) and the encoder output, generating another N embeddings.
- Each of the N embeddings goes through feed-forward layers predicting the bounding box (coordinates, width, height) and the object class — or that there is no object.

DETR: ENCODER-DECODER

- The encoder expects a sequence, so the backbone's feature map of size `[dimension, height, width]` is downsized and flattened to `[dimension, less than height` $\times$ `width]`.



Left: 12 of 256 feature-map dimensions; each extracts some features of the cat image (some focus on the cat's patterns, some on the bed sheets). Right: keeping 256 dimensions, width and height are flattened into size 850.

DETR: POSITIONAL EMBEDDINGS

- Transformers are permutation invariant, so positional embeddings are added in both encoder and decoder to remind the model where in the image an embedding comes from.
- **Encoder: fixed** positional encodings, as in the original Transformer — sinusoidal functions of varying frequencies at different feature dimensions. A sense of position without learned parameters.
- **Decoder: learned** positional encodings (the object queries) — each position has its own encoding, learned during training.

DETR: SET-BASED GLOBAL LOSS

- YOLO's loss: bounding box, objectness (probability that an object exists in a region of interest) and class loss, computed over a fixed number of boxes per grid cell.
- DETR generates unique bounding boxes in a **permutation-invariant** manner: the order of the detections does not matter, and the boxes must differ from each other. So predictions must be **matched** to the ground truth before they can be scored.
- **Bipartite matching**: a one-to-one matching between ground-truth and predicted boxes, pairing those with the highest similarity in box and class.
  - The closest prediction is matched with the corresponding ground truth, so boxes and classes are adjusted properly in the loss.
  - Without matching, a correct prediction that is not in the same order as the ground truth would be marked incorrect.

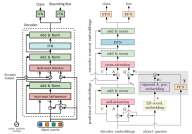
EVOLUTION OF DETR: DEFORMABLE DETR

Two main problems of DETR: long, slow convergence, and suboptimal detection of small objects.

- **Deformable attention** (fixes slow convergence): global attention is inefficient and heavily limits image resolution. Instead, the model attends only to a fixed number of *sampling points* around each *reference point*; the reference points are learned from the input.
  - E.g. in an image of a dog, a reference point may be at the dog's center, with sampling points near the ears, mouth, tail.
- **Multi-scale deformable attention** (fixes small objects): as in YOLOv3, multi-scale feature maps. Different layers of deformable attention give different resolutions; connecting some encoder layers' outputs to the decoder lets the model detect objects of many sizes.

EVOLUTION OF DETR: CONDITIONAL DETR

- Also targets slow convergence: over 6.7× faster. The object queries turn out to be general, not specific to the input image; **conditional cross-attention** lets the queries better localize the regions for box regression.
- Content query c_q (decoder self-attention output) and spatial query p_q: DETR adds them, Conditional DETR **concatenates** them — c_q focuses on the object's content, p_q on the box region.



Left: DETR decoder layer. Right: Conditional DETR decoder layer.

▸ Source: Meng et al., 2021

DETR: PYTORCH IMPLEMENTATION

```
class DETR(nn.Module):
    def __init__(self, num_classes, hidden_dim, nheads, num_encoder_layers, num_decoder_layers):
        super().__init__()
        self.backbone = nn.Sequential(*list(resnet50(pretrained=True).children())[:-2])
        self.conv = nn.Conv2d(2048, hidden_dim, 1)
        self.transformer = nn.Transformer(hidden_dim, nheads, num_encoder_layers, num_decoder_layers)
        self.linear_class = nn.Linear(hidden_dim, num_classes + 1)
        self.linear_bbox = nn.Linear(hidden_dim, 4)
        self.query_pos = nn.Parameter(torch.rand(100, hidden_dim))
        self.row_embed = nn.Parameter(torch.rand(50, hidden_dim // 2))
        self.col_embed = nn.Parameter(torch.rand(50, hidden_dim // 2))

    def forward(self, inputs):
        x = self.backbone(inputs)
        h = self.conv(x)
        H, W = h.shape[-2:]
        pos = torch.cat([self.col_embed[:W].unsqueeze(0).repeat(H, 1, 1),
                         self.row_embed[:H].unsqueeze(1).repeat(1, W, 1)],
                        dim=-1).flatten(0, 1).unsqueeze(1)
        h = self.transformer(pos + h.flatten(2).permute(2, 0, 1), self.query_pos.unsqueeze(1))
        return self.linear_class(h), self.linear_bbox(h).sigmoid()
```

The implementation from the original paper ▸ Carion et al., 2020, as shown in the HF course.

DETR: THE `forward` FUNCTION, LINE BY LINE

- **Backbone**: the image goes through a ResNet backbone, then a convolution reducing the dimension to `hidden_dim`.
- **Positional embeddings**: the paper uses fixed (encoder) and learned (decoder) embeddings; for simplicity the implementation uses learned ones for both (`nn.Parameter`). Row and column embeddings combined denote the location in the image.
- **Resize**: features of size `(batch, hidden_dim, H, W)` are reshaped to `(hidden_dim, batch, H*W)` — a sequential input for the Transformer.
- **Transformer**: the encoder input is the resized features plus positional embeddings; the decoder input is `query_pos`, the decoder positional embedding.
- **Feed-forward network**: two linear layers. The first predicts the class, with one extra "no object" class; the second predicts the bounding box (output size 4: x, y, height, width).

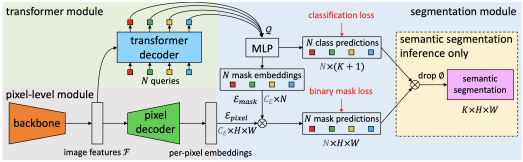
Transformer-Based Image Segmentation

CNNS VS. TRANSFORMERS FOR SEGMENTATION

Before ViTs, CNNs (U-Net, Mask R-CNN) were the go-to choice for segmentation. Limitations Transformers aim to solve:

- **Spatial limitations**: CNNs learn local patterns through small receptive fields, so it is hard to link features that are far apart but related. ViTs capture global dependencies via attention, considering the entire image at once.
- **Task-specific components**: Mask R-CNN uses hand-designed components (non-maximum suppression, spatial anchors) that add complexity and require manual tuning. ViT-based methods eliminate them.
- **Task specialization**: CNN-based models treat semantic, instance and panoptic segmentation separately. ViT-based models such as MaskFormer, SegFormer or SAM handle all three in a single framework.

MASKFORMER



Source: Cheng et al., 2021

- **Pixel-level module**: a backbone extracts image features; a pixel decoder generates per-pixel embeddings.
- **Transformer module**: a standard Transformer decoder computes per-segment embeddings from image features and learnable queries.
- **Segmentation module**: class predictions (linear classifier) and mask embeddings (MLP) per segment; mask embeddings combined with per-pixel embeddings give a binary mask per segment.

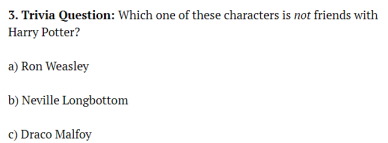
MASKFORMER: TRAINING AND FINE-TUNING

- MaskFormer predicts segmentation masks for each class present in an image, unifying semantic and instance segmentation in one architecture.
- Trained with a binary mask loss — the same one as DETR — plus a cross-entropy classification loss per predicted segment.
- **Panoptic segmentation**: label every pixel with its category *and* identify distinct objects within those categories (semantic + instance). MaskFormer's output contains multiple instances of the same class, each with its own mask.
- Transformer-based segmentation models are data-hungry and hard to train from scratch, so transfer learning is common: typically the backbone, the pixel decoder and the Transformer decoder are frozen, and the rest is fine-tuned for new class predictions and masks.

Knowledge Distillation with Vision Transformers

KNOWLEDGE DISTILLATION: INTUITION

- Instead of training a model on labels, **knowledge distillation** arrives at a much smaller model that performs comparably to a larger one — and is much faster. It is the method behind DistilBERT and DistilGPT.


3. Trivia Question: Which one of these characters is *not* friends with Harry Potter?

a) Ron Weasley

b) Neville Longbottom

c) Draco Malfoy

- Being told "the answer is Draco Malfoy" teaches you little about the characters' relationships to Harry Potter.
- "I am very confident it is *not* Ron, somewhat confident it is not Neville, and very confident it *is* Draco" carries much more information. This is the information passed to the student model.

DISTILLING THE KNOWLEDGE IN A NEURAL NETWORK

- Introduced by Hinton et al. ▸ Hinton et al., 2015, inspired by insects: just as insects move from larval to adult forms optimized for different tasks, a cumbersome large model good at extracting structure from data can distill its knowledge into a smaller model for deployment.
- The teacher's predicted logits pass information to the student via a **distillation loss**, which encourages the student's output distribution to approximate the teacher's:

$$L_{\text{distillation}} = \mathrm{KL}\big(\sigma(z_t;\, T = t),\, \sigma(z_s;\, T = t)\big)$$

  with teacher logits z_t, student logits z_s, softmax σ at temperature T, and the Kullback-Leibler divergence KL.
- The student's overall loss: distillation loss + standard cross-entropy loss on the ground-truth labels.

DISTILLATION FOR EDGE DEVICES

- Deploying a large model (e.g. 1 GB, 1 s latency) is impractical for real-time applications on edge devices. Distillation can reduce parameters by over 90% with minimal performance degradation.
- **Consequences (good and bad):**
  1. **Entropy gain**: which is harder to remember, `[0, 1, 0, 0]` or `[0.2, 0.5, 0.2, 0.1]`? The second contains more information. A distilled student's outputs have higher entropy than its teacher's: even when correct, they are more "chaotic", since the teacher's extra parameters help it discern between classes.

DISTILLATION FOR EDGE DEVICES (2)

- **Consequences (continued):**
  2. **Coherent gradient updates**: a misclassified ambiguous sample causes an aggressive update; the teacher's soft targets make updates more stable and coherent.
  3. **Training on unlabeled data**: the teacher can produce pseudo-labels for unlabeled samples.
  4. **A shift in perspective**: instead of approximating the unknown function F from scarce data, approximate a well-trained teacher F with a smaller model f.

Introduction to Vision Language Models

OUR WORLD IS MULTIMODAL

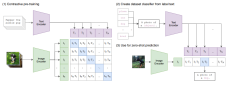
- Humans explore the world through diverse senses: sight, sound, touch, scent. Machine learning seeks models that process and integrate data across multiple **modalities**.
- Two key challenges:
  - **Representation**: effectively summarizing multimodal data, capturing the connections among individual modality elements.
  - **Alignment**: identifying connections and interactions across all elements.
- Inherent difficulties: one modality may dominate the others; additional modalities can introduce noise; full coverage over all modalities is not guaranteed; modalities can have complicated relationships.
- This chapter: the fusion of vision and language — **Vision Language Models (VLMs)**.

FROM TRAINING FROM SCRATCH TO PRE-TRAINING

- Generating text from images (captioning, visual question answering) has been studied for years.
- A shift from training from scratch to the paradigm first proposed in ULMFiT ▸ Howard and Ruder, 2018:
  - pre-train the model with extensive training data,
  - fine-tune it with task-specific data,
  - use it for downstream tasks such as classification.
- VLMs extend this paradigm to images. In 2021, OpenAI's **CLIP** used an image-text **contrastive** objective: paired images and texts are brought close in the embedding space, all others pushed apart.
- This captures rich vision-language correspondence, enabling **zero-shot** predictions by matching embeddings of any image and text. CLIP outperformed ImageNet models on out-of-distribution tasks.

VLM MECHANISM

- Given image-text pairs: extract features with a text encoder and an image encoder (CNN- or Transformer-based), then learn the vision-language correlation with a pre-training objective:
  - **contrastive**: pull paired samples close, push others far away in the embedding space;
  - **generative**: learn semantic features by generating image/text data;
  - **alignment**: global image-text matching or local region-word matching.
- With the learned correlation, VLMs are evaluated zero-shot by matching embeddings of any given images and texts.



▸ Source: Radford et al., 2021

VLM STRATEGIES

Grouping VLMs by how they combine the two modes of learning:

- **Images as tokens**: divide images into patches, treat each patch as a "token", and train on joint sequences of text and image tokens (VisualBERT, SimVLM).
- **Image embeddings as a prefix** for a *frozen* pre-trained language model: learn an image embedding space compatible with the language model's, without changing its parameters (Frozen, ClipCap).
- **Cross-attention**: a specially designed cross-attention mechanism fuses visual information into the layers of the language model (VisualGPT).
- **No training**: combine vision and language models via guided decoding of the next token (MAGiC).

Research focus: collecting large-scale informative image-text data, designing effective models for big data, and new pre-training objectives.

COMMON VLM DATASETS

- **MSCOCO**: 328K images, each paired with 5 independent captions.
- **NoCaps**: measures generalization to unseen classes and concepts (in-domain, near-domain, out-of-domain).
- **Conceptual Captions**: 3 million image-caption pairs, mined from the web and post-processed.
- **ALIGN**: a noisy dataset of over one billion image alt-text pairs, without expensive filtering or post-processing.
- **LAION**: image-text pairs extracted from Common Crawl (web pages crawled 2014–2021); used to train Stable Diffusion.

DOWNSTREAM TASKS AND EVALUATION

- Downstream tasks: image classification, object detection, semantic segmentation, image-text retrieval, action recognition.
- **Zero-shot prediction** (most common): apply the pre-trained VLM directly, without task-specific fine-tuning.
- **Linear probing**: freeze the VLM and train a linear classifier on its embeddings, measuring the quality of the representation.
- Harder datasets probe generalization: **Hateful Memes** adds difficult "benign confounders"; **Winoground** tests whether models truly grasp compositional relationships.



(a) some plants surrounding a lightbulb

(b) a lightbulb surrounding some plants

Figure 1. An example from Winoground. The two sentences contain the same words but in a different order. The task of understanding which image and caption match is trivial for humans but much harder for vision and language models. Every model that we tested (UNITER, ViLLA, VinVL, VisualBERT, ViLT, LXMERT, ViLBERT, UniT, FLAVA, CLIP, VSE++, and VSRN) fails to correctly pair the images and captions, except the large checkpoint of ViLLA by a very thin margin (0.00013 confidence).

▸ Source: Thrush et al., 2022

CLIP and Relatives

BEFORE AND AFTER CLIP

Pre-CLIP:

- *Multimodal Deep Learning* (Ngiam et al., 2011): deep learning for multimodal inputs.
- *Deep Visual-Semantic Alignments* (Karpathy & Fei-Fei, 2015): aligning text with specific image regions.
- *Show and Tell* (Vinyals et al., 2015): CNNs and RNNs combined to turn images into descriptive language.

Post-CLIP:

- **CLIP**: learns from a vast array of internet image-text pairs; zero-shot learning.
- **GroupViT**: segmentation and semantic understanding combined with language.
- **BLIP**: bidirectional learning between vision and language; generating text from images.
- **OWL-ViT**: object-centric representations, understanding objects in images in context with text.

Losses

CONTRASTIVE LEARNING

- **Contrastive learning**: an unsupervised method for representation learning.
- Objective: a representation in which similar items are positioned close together and dissimilar items are clearly separated.
- Example: keep the representations of dogs close to other dogs, but far from cats.



CONTRASTIVE LOSS

- One of the first training objectives for contrastive learning. Input: a pair of samples, similar or dissimilar. Goal: map similar samples close in the embedding space, push dissimilar ones apart.
- For samples x_i, x_j with classes y_i, y_j:

$$L = \mathbb{1}[y_i = y_j] \, \|x_i - x_j\|^2 + \mathbb{1}[y_i \neq y_j] \, \max(0, \epsilon - \|x_i - x_j\|^2)$$

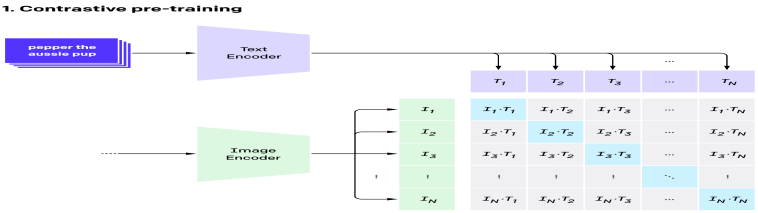
- If the samples are similar ($y_i = y_j$): minimize their squared Euclidean distance — make them closer.
- If they are dissimilar ($y_i \neq y_j$): minimizing $\max(0, \epsilon - \|x_i - x_j\|^2)$ means maximizing their distance, up to a margin ϵ.

Contrastive Language-Image Pre-training (CLIP)

CLIP

- CLIP ▸ Radford et al., 2021 grasps visual concepts through **natural language supervision**: a text encoder and an image encoder are trained jointly on matching captions with their images.
- It adapts to a variety of visual classification benchmarks by simply receiving the *names* of the visual categories to be recognized — "zero-shot" capabilities akin to those of GPT-2 and GPT-3.

CONTRASTIVE PRE-TRAINING



▸ Source: OpenAI

- Given a batch of image-text pairs, compute the cosine similarity matrix between all possible (image, text) candidates.
- Maximize the similarity of correct pairs (blue diagonal), minimize it for incorrect pairs, via a **symmetric cross-entropy loss** over the similarity scores.
- In simple terms: each image should be most similar to its own caption, and each caption to its own image.

CLIP: ENCODERS, USE CASES, LIMITATIONS

- **Independent encoders** for images and text: e.g. a Vision Transformer or a ResNet as image encoder. Switching an encoder requires retraining, as the embedding distribution changes.
- **Use cases**: zero-shot image classification, similarity search, conditioning diffusion models.
- **Limitations**:
  - Unlikely to outperform a specialized, fine-tuned model; generalization is limited for data not encountered during training.
  - Effectiveness and biases depend on the choice of categories: on the FairFace dataset, gender accuracy was over 96%, racial accuracy around 93%.

CLIP: ZERO-SHOT CLASSIFICATION IN PRACTICE

```
from transformers import CLIPProcessor, CLIPModel

model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

inputs = processor(text=["a photo of a cat", "a photo of a dog"],
                   images=image, return_tensors="pt", padding=True)
outputs = model(**inputs)
probs = outputs.logits_per_image.softmax(dim=1)
```

- Zero-shot classification of a photo of cats between `dog` and `cat`: the class names become text prompts, and the image is matched against them.
- Result: “a photo of a cat” 99.49%, “a photo of a dog” 0.51%.

Multimodal Text Generation (BLIP)

BLIP: BOOTSTRAPPING LANGUAGE-IMAGE PRE-TRAINING

- BLIP ▸ Li et al., 2022 extends multimodal models to **text generation**.
- **CapFilt (caption and filtering)**: web-scraped image/alt-text pairs are noisy — alt-texts often do not describe the image. One model filters out noisy pairs, another creates captions for images; both are first fine-tuned on a human-annotated dataset. Cleaning the data this way beat training on the raw web data.
- Takeaway: dataset **quality** matters, not just quantity.
- Successor: BLIP-2 ▸ Li et al., 2023, e.g. for visual question answering.

BLIP: ARCHITECTURE

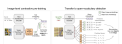
A vision encoder plus a **Multimodal mixture of Encoder-Decoder (MED)**:

- **Vision Transformer**: a plain ViT with a `[CLS]` token for the embedding.
- **Unimodal text encoder**: like BERT, with a `[CLS]` token; trained with a contrastive loss like CLIP to align image and text representations.
- **Image-grounded text encoder**: `[CLS]` replaced by an `[Encode]` token; cross-attention layers combine image and text embeddings into a multimodal representation; a linear layer judges whether image and text match.
- **Image-grounded text decoder**: bidirectional self-attention replaced by **causal** self-attention; trained autoregressively with cross-entropy, for caption generation or visual question answering.

Multimodal Object Detection (OWL-ViT)

OWL-VIT: OPEN-VOCABULARY OBJECT DETECTION

- Detectors like YOLO cannot detect objects outside their training datasets. OWL-ViT ▸ Minderer et al., 2022 is akin to CLIP, but for object detection.
- **Pre-training**: like CLIP, a vision and a language encoder with a contrastive loss — a shared representation space for images and text.
- **Fine-tuning for detection**: instead of CLIP's token pooling and final projection, each output token is linearly projected to a per-object image embedding used for classification; box coordinates come from the token representations via a small MLP.



▸ Source: Minderer et al., 2022

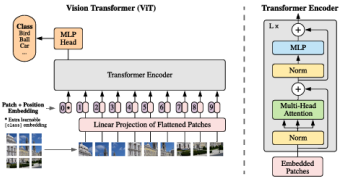
OWL-VIT: EXAMPLE

- Thanks to the shared embedding space, OWL-ViT identifies objects not present in its training data; both images and text can be used as queries.
- Example: text queries for objects likely in detection datasets (“cat”) and more abstract ones (“cat tail”) that most likely are not.



Transformers in Video Processing

RECAP: VIT



▸ Source: Dosovitskiy et al., 2020

- Images are divided into small patches; each patch is an input to a Transformer, replacing CNNs.
- Each patch is linearly mapped, and positional embeddings are added so the Transformer can recognize the order of the patches.
- Pre-trained on large-scale datasets and fine-tuned for downstream vision tasks.

VIT: PERFORMANCE AND LIMITATION

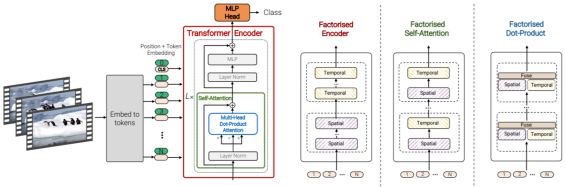
	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	**88.55** ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	**90.72** ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	**99.50** ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	–
CIFAR-100	**94.55** ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	–
Oxford-IIIT Pets	**97.56** ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	–
Oxford Flowers-102	99.68 ± 0.02	**99.74** ± 0.00	99.61 ± 0.02	99.63 ± 0.03	–
VTAB (19 tasks)	**77.63** ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	–
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

▸ Source: Dosovitskiy et al., 2020

- ViT outperformed other state-of-the-art models, but training it required 2,500 TPU-v3 days. At roughly \$2 per TPU-v3 core-hour, one training run costs about $2 \times 24 \times 2500 =$ \$120,000.

VIDEO VISION TRANSFORMER (VIVIT)

- ViViT ▸ Arnab et al., 2021 extends ViT to video classification. Key issues: train more quickly and efficiently, and handle **temporal** as well as spatial information.
- Extracts spatio-temporal tokens from the video and encodes them with Transformer layers; efficient variants factorise the spatial and temporal dimensions; pre-trained image models make training on comparatively small datasets possible.

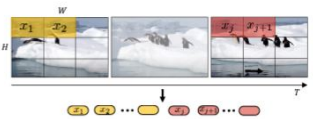


▸ Source: Arnab et al., 2021

EMBEDDING VIDEO CLIPS

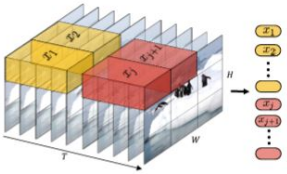
- Embeddings: dense vectors capturing meaningful features. For video, they must preserve **spatial** (what is in each frame) and **temporal** (how things change) information, while reducing dimensionality so processing stays feasible.

Uniform frame sampling



Sample frames uniformly over time, e.g. one in every 2 frames.

Tubelet embedding



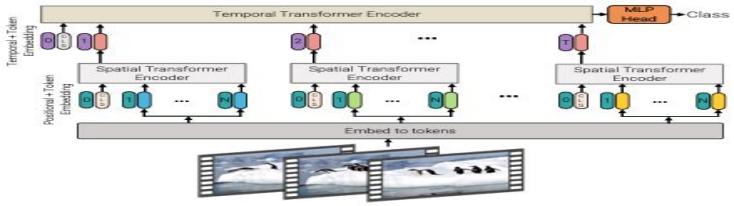
Extract spatio-temporal "tubes" and project them linearly: fuses space and time already during tokenization.

Figures: ▸ Arnab et al., 2021

VIVIT MODEL 1: SPATIO-TEMPORAL ATTENTION

- The natural extension of ViT: each of the n_t frames is split into $n_w \times n_h$ patches, giving $n_t \times n_w \times n_h$ **spatio-temporal tokens**.
- All tokens go through the Transformer encoder together, so each token is contextualized with all others across space and time — patterns in position, color and movement.
- Complexity: $\mathcal{O}(n_h^2\, n_w^2\, n_t^2)$.
- Attention over all spatio-temporal tokens is expensive; uniform frame sampling and tubelet embedding help reduce the cost.

VIVIT MODEL 2: FACTORISED ENCODER

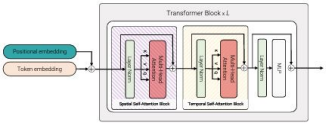


▸ Source: Arnab et al., 2021

- Model 1 contextualizes all patches simultaneously — inefficient. Model 2 separates spatial and temporal encoders, sequentially:
  - a **spatial** Transformer encoder (= ViT) contextualizes within each frame and encodes it into a single embedding;
  - a **temporal** Transformer encoder then processes the frame embeddings.
- Complexity: $\mathcal{O}(n_h^2\, n_w^2 + n_t^2)$.

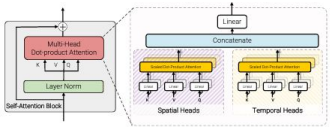
VIVIT MODELS 3 AND 4

Model 3: Factorised Self-Attention



First self-attention **spatially** (among tokens of the same time index), then **temporally** (among tokens of the same spatial index). No `[CLS]` token, because of ambiguities.

Model 4: Factorised Dot-Product Attention



Half of the attention heads use keys and values from the same spatial indices, the other half from the same temporal indices.

Complexity of both: same as Model 2. Figures: ▸ Arnab et al., 2021

ViViT: Experiments and Discussion

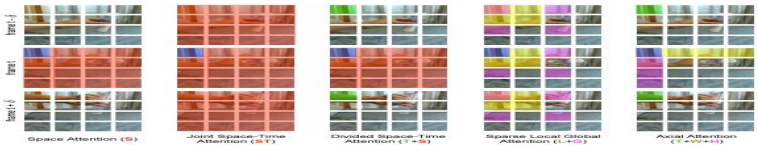
	K400	EK	FLOPs ($\times 10^9$)	Params ($\times 10^6$)	Runtime (ms)
Model 1: Spatio-temporal	80.0	43.1	455.2	88.9	58.9
Model 2: Fact. encoder	78.8	43.7	284.4	115.1	17.4
Model 3: Fact. self-attention	77.4	39.1	372.3	117.3	31.7
Model 4: Fact. dot product	76.3	39.5	277.1	88.9	22.9
Model 2: Ave. pool baseline	75.8	38.8	283.9	86.7	17.3

▸ Source: Arnab et al., 2021

- Model 1 achieves the best accuracy but needs the longest training; Model 2 gives relatively high accuracy with shorter training than Models 3 and 4 — the most efficient overall.
- Like ViT, ViViT needs extremely large datasets, which rarely exist for video. Approach: first pre-train on a large image dataset with ViT to initialize the model.

TIMESFORMER

Concurrent work with ViViT (Bertasius et al., 2021); five space-time self-attention schemes (blue: query patch):



- **Space**: as in ViT, within one frame. **Joint space-time**: across frames, as ViViT Model 1.
- **Divided space-time**: first temporally (same position, other frames), then spatially — similar to ViViT Model 3.
- **Sparse local global**: selectively combines local and global information. **Axial**: spatial and temporal dimensions separately along their axes.
- **Divided space-time attention** gives the best balance of parameter efficiency and accuracy (K400, SSv2).

CONCLUSION: VIDEO TRANSFORMERS

- ViViT extended ViT to video with several models (factorised encoder, factorised self-attention, factorised dot-product attention), all aimed at managing the space-time dimensions efficiently.
- TimeSformer likewise evolved from ViT, using diverse attention mechanisms over space and time.
- Key takeaway: the focus is on reducing the significant computational cost of applying Transformers to video, enabling learning with fewer computational resources.

License & Attribution

LICENSE & ATTRIBUTION

- **Single source.** All content follows the **Hugging Face Community Computer Vision Course** (`https://huggingface.co/learn/computer-vision-course`, repository `https://github.com/johko/computer-vision-course`, © 2023 Johannes Kolbe and contributors) — licensed **MIT**. Units 3 (*Vision Transformers*), 4 (*Multimodal Models*) and 7 (*Video*), in the course's own order. The course is prose, not slides: frames condense its text into bullet points without adding content.
- **Figures** are those the course uses, with the attribution the course gives: most are taken from the original papers (linked on each frame); the CLIP pre-training figure is from OpenAI; the MaskFormer and OWL-ViT figures come via the Hugging Face documentation.